

Data Structures and Algorithms (CS1.201)

End-Sem Exam (2 May, 2026)

Total Marks: 60 [14+12+9+13+12]

Instructors: Lini Thomas, Kshitij Gajjar, Raghavendra G. S.

ATTENTION! Please check that this question paper has 5 puzzles, which add to 60 marks total. All the best! ☺

Puzzle 1 [3+3+4+4=14 marks]. [For this puzzle, assume that $n \geq 8$ and n is a power of 2; that is, $n = 2^k$, where $k \geq 3$ is an integer.]

Suppose you have a function `THREEFOURTHSORT`, which can sort $3n/4$ distinct numbers. You don't know the details about the inner workings of `THREEFOURTHSORT`, but you know it works correctly. You wish to sort n numbers using `THREEFOURTHSORT`, without doing any comparison (other than comparisons done within `THREEFOURTHSORT` itself).

Let A be an array of n distinct numbers, indexed from $A[0]$ to $A[n-1]$. Consider the following algorithm (let's call it Algorithm 1): sort the first $3n/4$ entries of A , and then sort its last $3n/4$ entries (both using `THREEFOURTHSORT`), as shown below. `THREEFOURTHSORT` takes three arguments, the first being the array, and the other two being the starting and ending indices of the sub-array of size $3n/4$ which you want to sort.

Algorithm 1:

`THREEFOURTHSORT(A, 0,  $3n/4 - 1$ )`

`THREEFOURTHSORT(A,  $n/4$ ,  $n - 1$ )`

1 2 3 4 5 6 7 8

(i) Provide a counterexample to show that Algorithm 1 doesn't necessarily sort n distinct numbers.

Next, consider the following algorithm (let's call it Algorithm 2): replicate Algorithm 1, and after that sort the first $3n/4$ entries of the array A once again using `THREEFOURTHSORT`, as shown below.

Algorithm 2:

`THREEFOURTHSORT(A, 0,  $3n/4 - 1$ )`

`THREEFOURTHSORT(A,  $n/4$ ,  $n - 1$ )`

`THREEFOURTHSORT(A, 0,  $3n/4 - 1$ )`

(ii) Prove that Algorithm 2 sorts n distinct numbers. You may use a helpful example to carry along your proof.

Now, suppose you have a function `HALFSORT`, which can sort $n/2$ distinct numbers. You don't know the details about the inner workings of `HALFSORT`, but you know it works correctly. You wish to sort n numbers using `HALFSORT`, without doing any comparison (other than comparisons done within `HALFSORT` itself).

`HALFSORT` takes three arguments, the first being the array, and the other two being the starting and ending indices of the size $n/2$ sub-array which you want to sort. Consider the following algorithm (let's call it Algorithm 3): sort the first half and second half of A . Then, sort the middle half of A . Finally, repeat these once.

Algorithm 3:

`HALFSORT(A, 0,  $n/2 - 1$ )`

`HALFSORT(A,  $n/2$ ,  $n - 1$ )`

`HALFSORT(A,  $n/4$ ,  $3n/4 - 1$ )` ✓

`HALFSORT(A, 0,  $n/2 - 1$ )` ✓

`HALFSORT(A,  $n/2$ ,  $n - 1$ )`

`HALFSORT(A,  $n/4$ ,  $3n/4 - 1$ )`

$n/4 = 2$

(iii) Prove that Algorithm 3 sorts n distinct numbers. You may use a helpful example to carry along your proof.

$\frac{3 \times 8}{8} = 3$

8 7 5 6 4 3 2 1
3 4

Thus, as we have seen, n distinct numbers can be sorted by making six calls to a function that sorts $n/2$ of them, without doing any comparison apart from comparisons made in those six calls. So, suppose you construct a function SORT2, which sorts an array of size two, which can obviously be done by just making one comparison. Using this, you construct a function SORT4 by calling the function SORT2 six times. Using SORT4, you construct SORT8 by calling SORT4 six times, and so on.

Let A be an array of size $n = 2^k$. Start by pairing up consecutive elements ($A[0]$ and $A[1]$ is the first pair, $A[2]$ and $A[3]$ is the second pair, and so on), and call the function SORT2 for each pair. This makes $n/2$ comparisons as there are $n/2$ such pairs. Then, make groups of size four ($A[0], A[1], A[2], A[3]$ is the first group, and so on), and call the function SORT4 for each group, which calls SORT2 six times for each group. This makes $6 \times n/4$ comparisons, since SORT4 doesn't make any comparisons apart from comparisons made in those six calls. Similarly, there are $n/8$ groups of size eight ($A[0]$ to $A[7]$ is the first group, and so on). Each group calls SORT4 six times, doesn't make any extra comparisons, and thus does $6 \times n/8$ comparisons. Eventually, there will be only one group of size n ($A[0]$ to $A[n-1]$), and the entire array is sorted!

Let's now calculate C_n , the total number of comparisons made to sort n distinct numbers.

$$\begin{aligned} C_n &= \frac{n}{2} + \frac{6n}{4} + \frac{6n}{8} + \dots + \frac{6n}{n} \\ &\leq \frac{6n}{2} + \frac{6n}{4} + \frac{6n}{8} + \dots + \frac{6n}{n} \\ &\leq 6n \left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \frac{1}{n} \right) \\ &\leq 6n \left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \frac{1}{2^k} \right) \\ &\leq 6n. \end{aligned}$$

Thus, $C_n \leq 6n$, implying that n distinct numbers can be sorted by making at most $6n$ comparisons.

- (iv) Is it really true that n distinct numbers can be sorted using just $O(n)$ comparisons? If not, then point out the exact place where the flaw lies in the above analysis.

Puzzle 2 [3+3+3+3=12 marks]. Consider insertion and deletion operations in an AVL tree.

Let Z be the lowest (that is, closest to the point of insertion/deletion) node on the path from the point of update to the root that becomes unbalanced. Let Y be the child of Z , and X be the child of Y used in AVL tree rebalancing.

For each of the following statements, determine whether it is True or False, and provide a clear justification.

If the statement is false, you must also provide a specific counterexample. Your reasoning must explicitly refer to subtree heights and balance factors where applicable. Use diagrams where appropriate to support your explanation.

- Deletion of a node does not necessarily result in an unbalanced node. Justify your answer by analyzing how the relative heights of the left and right subtrees of node Z before deletion determine whether an imbalance occurs.
- In case of node deletion, the node Y could be either of the children of Z .
- In case of node deletion, if the left and right subtrees of Y have the same height, then X is uniquely determined and can not be either of the children of Y .
- In case of node insertion, the first unbalanced node Z could be the parent of a leaf. In such a case, Z will have a child Y on the path of insertion but may not have a grandchild X on the path from Z toward the inserted node.

Puzzle 3 [3+3+3=9 marks]. As studied in class, we represent binary trees as a data structure using pointers, wherein each node has one pointer to its parent node (denoted by $\text{node} \rightarrow \text{parent}$), one pointer to its left child (denoted by $\text{node} \rightarrow \text{left}$), and one pointer to its right child (denoted by $\text{node} \rightarrow \text{right}$).

If a node is a leaf node, then its left and right child pointers are both null pointers. If a node has exactly one child, then its other child pointer is a null pointer. The root node is the only node whose parent pointer is a null pointer.

Let B be a binary tree. Consider the variables n, ℓ, x, i_1, i_2 , defined as follows.

- n : number of nodes in B (assume $n \geq 1$).
- ℓ : number of leaf nodes in B .
- x : number of **null** pointers in the data structure representation of B using pointers.
- i_1 : number of nodes in B with exactly one child (its other child pointer is **null**).
- i_2 : number of nodes in B with exactly two children (none of its child pointers is **null**).

For each of the following statements, determine whether it is true or false, providing a clear justification. If it is true, provide a proof. If it is false, provide a counterexample. Diagrams are encouraged where helpful. Answers without reasoning will not receive any credit.

- $x = 2i_2 + i_1 + 3$, for all binary trees B .
- $x = 2\ell + 1$, if every node in B has an even number of descendants (note that zero is also an even number).
- $x = n + 2$, for all binary trees B .

Puzzle 4 [3+4+4+2=13 marks]. Assume that you are in charge of managing the traffic in your city. All the streets are two-way streets, and you bring in an order to making every street one-way.

Initially, the city is connected (with two-way streets), which means that you can reach from any point in the city to any other point. Your aim is to make all the streets one-way in such a manner that you can still reach from any point to any other point.

Devise an algorithm (pseudocode is sufficient) to make all the streets one-way. If reachability from every point to every other point cannot be maintained after making all streets one-way, then your algorithm should output "IMPOSSIBLE". Your solution will be evaluated on the first three points below (parts (i), (ii), (iii)):

- Pseudocode: convert this task into an algorithmic problem, stating clearly how you are modelling it, and then write the pseudocode for solving it. You are allowed to use the algorithms (such algorithms can be referred to as a subfunction and don't need elaboration) and data structures taught in class.
- Running time: analyze the worst-case running time of your algorithm (the lower the running time, the better).
- Explanation of why it works: provide a detailed justification of all the steps of your algorithm (your proof should be clear, correct, coherent, and complete).
- What initial condition(s) should the original two-way network of streets satisfy so that reachability can be maintained from every point to every other point after making all the streets one-way? (Proof not needed.)

Puzzle 5 [2+2+2+2+2=12 marks]. As we studied in class, the Floyd-Warshall algorithm computes All-Pairs Shortest Paths (APSP) in a given directed graph. The algorithm has three nested loops, but it is easy to mix up to ordering of the variables i, j, k , leading to incorrect implementations of the algorithm. Let's see how.

Consider the following six algorithms. The input to each algorithm is a directed, weighted graph G (with weight function wt) on n vertices whose vertices are labelled $\{0, 1, 2, \dots, n-1\}$. A two-dimensional array $dist$ of size $n \times n$ is initialized as follows. For every $0 \leq i \leq n-1$ and $0 \leq j \leq n-1$:

- If (i, j) is an edge of G , then initialize $dist[i][j] = wt(i, j)$.
- If $i = j$, then initialize $dist[i][i] = 0$.
- Otherwise, initialize $dist[i][j] = \infty$.

Algorithm 1:

```
foreach (i from 0 to n - 1)
  foreach (j from 0 to n - 1)
    foreach (k from 0 to n - 1)
      if (dist[i][j] > dist[i][k] + dist[k][j])
        dist[i][j] = dist[i][k] + dist[k][j].
```

Algorithm 2:

```
foreach (i from 0 to n - 1)
  foreach (k from 0 to n - 1)
    foreach (j from 0 to n - 1)
      if (dist[i][j] > dist[i][k] + dist[k][j])
        dist[i][j] = dist[i][k] + dist[k][j].
```

Algorithm 3:

```
foreach (j from 0 to n - 1)
  foreach (i from 0 to n - 1)
    foreach (k from 0 to n - 1)
      if (dist[i][j] > dist[i][k] + dist[k][j])
        dist[i][j] = dist[i][k] + dist[k][j].
```

Algorithm 4:

```
foreach (j from 0 to  $n - 1$ )
  foreach (k from 0 to  $n - 1$ )
    foreach (i from 0 to  $n - 1$ )
      if ( $\text{dist}[i][j] > \text{dist}[i][k] + \text{dist}[k][j]$ )
         $\text{dist}[i][j] = \text{dist}[i][k] + \text{dist}[k][j]$ .
```

Algorithm 5:

```
foreach (k from 0 to  $n - 1$ )
  foreach (i from 0 to  $n - 1$ )
    foreach (j from 0 to  $n - 1$ )
      if ( $\text{dist}[i][j] > \text{dist}[i][k] + \text{dist}[k][j]$ )
         $\text{dist}[i][j] = \text{dist}[i][k] + \text{dist}[k][j]$ .
```

Algorithm 6:

```
foreach (k from 0 to  $n - 1$ )
  foreach (j from 0 to  $n - 1$ )
    foreach (i from 0 to  $n - 1$ )
      if ( $\text{dist}[i][j] > \text{dist}[i][k] + \text{dist}[k][j]$ )
         $\text{dist}[i][j] = \text{dist}[i][k] + \text{dist}[k][j]$ .
```

Answer the following questions regarding the correctness of the above algorithms, providing a justification for each. (If the algorithm is correct, then give a proof. If the algorithm is incorrect, then give a counterexample.)

- (i) Algorithm 1 correctly computes APSP in G .
 - (ii) Algorithm 2 correctly computes APSP in G .
 - (iii) Algorithm 3 correctly computes APSP in G .
 - (iv) Algorithm 4 correctly computes APSP in G .
 - (v) Algorithm 5 correctly computes APSP in G .
 - (vi) Algorithm 6 correctly computes APSP in G .
-